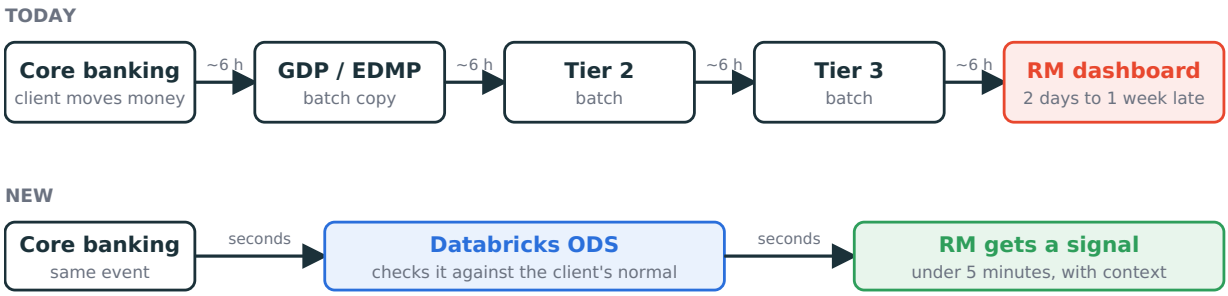
DATABRICKS × STANDARD CHARTERED BANK · WEALTH & RETAIL BANKING

Real-Time Money Movement Tracking for RMs

Visual implementation plan, business case and deployment guide. Written so anyone on the WRB, Architecture, Kafka, Database or Databricks teams can understand the project and deploy their part.

What problem are we solving?

When a wealth client moves a large amount of money (for example a 400K deposit or withdrawal), the Relationship Manager (RM) finds out two days later in Hong Kong and Singapore, and about a week later in India. By then the money has been spent, invested elsewhere, or, if it was a scam, is gone. The delay comes from the data passing through four batch systems, each adding roughly six hours.



What are we building?

A WRB-owned **Operational Data Store (ODS)** on Databricks that listens to the bank's existing real-time event feed, compares each movement to the client's own 30-day normal, filters out noise (do-not-contact, open complaint, already called three times), and gives the RM a signal with context within minutes. Signals also flow back to the bank's on-premise systems so existing applications can use them. Nothing about GDP/EDMP changes.

< 5 min
Target time from money movement to RM signal (Habeeb)

< 1 min
Measured end-to-end in the demo (P50 / P95 tested)

0
Clear-text customer identifiers stored in the cloud (tokenized before leaving on-prem)

13 wks
Indicative plan to a live pilot in HK / SG

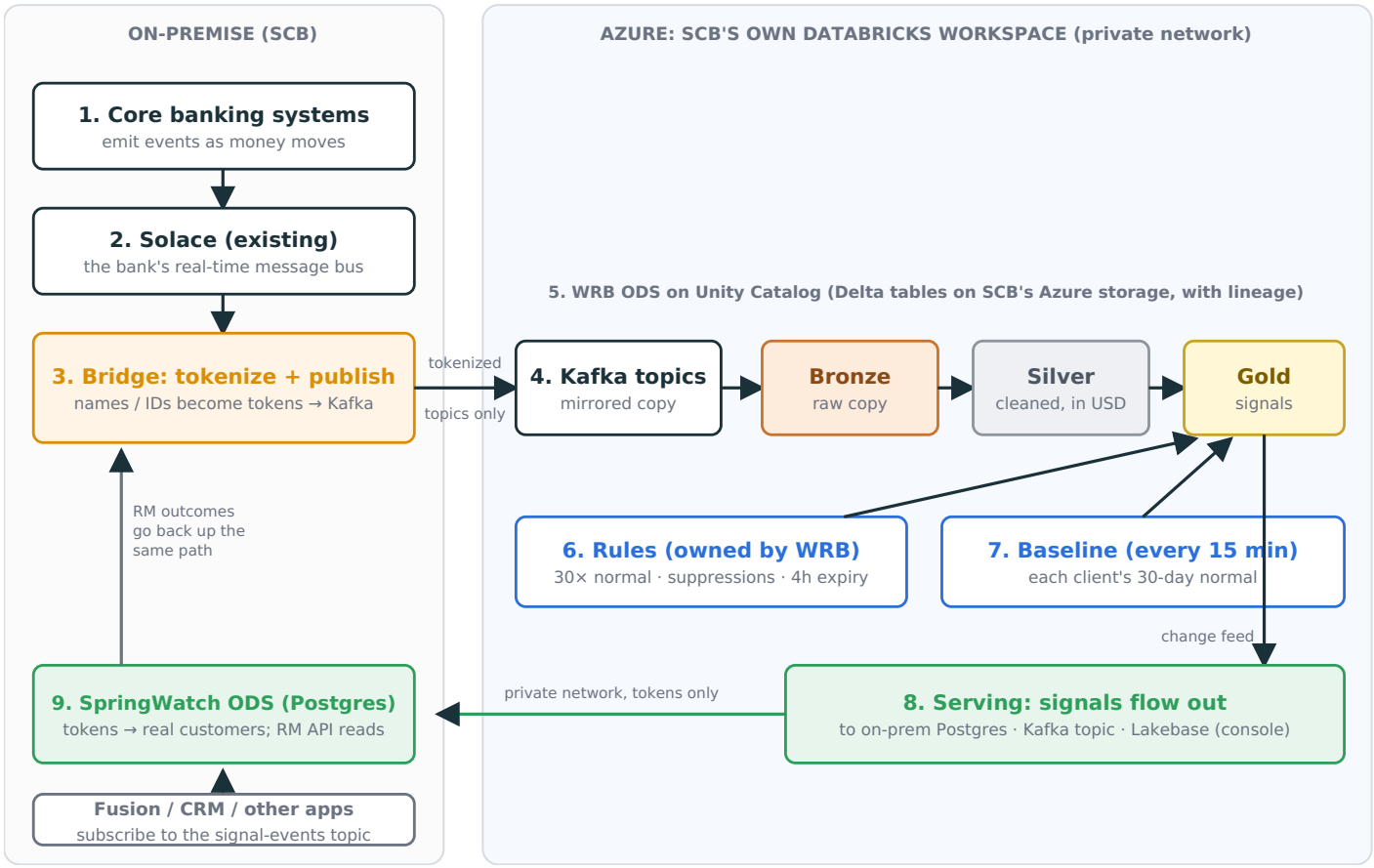
Who does what

TEAM	YOUR PART OF THE BUILD	PAGES FOR YOU
Kafka / Integration (SCB)	Deploy the bridge that reads Solace, tokenizes customer data and mirrors topics to Azure	5, 7, 9
Databricks (Databricks + WRB engineers)	Deploy the Bronze / Silver / Gold pipelines and the serving jobs with one bundle command	3, 8, 9
Database (SCB DBA)	Create the serving schema on the on-prem Postgres ODS and populate the token map	5, 8, 9
Architecture (Habeeb, Dhas, Mario)	Pattern approval, network path, persistence decision	4, 6, 10

THE BIG PICTURE

How the solution fits together

Follow the numbers. Everything to the left of the dotted line stays exactly where it is today; the new pieces are the bridge (3) and the Databricks ODS (4–8).



Bronze = the raw copy
Every event exactly as received, so we can always prove where a signal came from. Nothing is changed here.

Silver = the clean version
Duplicates removed, fields typed, amounts converted to USD so one set of rules works for HK, SG and India.

Gold = the answers
Which clients moved unusually large amounts, whether the RM should be told, why or why not, and when the signal expires.

Release 2 (later, optional): Zerobus lets source systems write straight into Databricks without a Kafka hop. Dhas's view is that it means changing the source teams' Solace client, so it is evaluated separately and is not needed for Release 1.

THE BUSINESS CASE

Why run the ODS and its data processing on Databricks?

WRB is already building an on-premise ODS on Postgres. This is not a replacement for it. The question is where the *processing* should happen: the streaming, the 30-day baselines, the rules, and the AI that follows. Here is the case for Databricks, in plain terms.

WHAT MATTERS	DOING IT ON DATABRICKS	DOING IT ON-PREMISE
Time to first signal	Weeks. Streaming, storage, SQL, rules and AI are one platform; the pipelines in this plan already exist and deploy with one command.	Months. Separate streaming cluster, database tuning, scheduler, monitoring, and later a separate AI stack, each to be procured, built and integrated.
Keeping up at peak	Compute grows and shrinks automatically. Month-end or a market event does not need a capacity request.	Sized for the peak and paid for all year, or undersized and late exactly when it matters.
One engine, not six	Ingest, clean, aggregate, serve, query in natural language and run AI models on the same tables. No copies between tools.	Each capability is a different product with its own copy of the data and its own team.
Proof of where a signal came from	Unity Catalog records lineage automatically: signal → Gold → Silver → Bronze → Kafka offset. Auditors and Dhas can see it without asking an engineer.	Lineage is documentation someone has to write and keep current.
Who can see what	Row and column permissions, audit logs and data classification are built in and apply to every tool reading the data.	Re-implemented per application; easy to drift.
Running it day to day	No servers to patch, no Spark or Postgres upgrades to plan. Jobs restart themselves; alerts are part of the deployment.	A platform team owns patching, upgrades, high availability and on-call.
Cost	Pay for compute while it runs. Streaming for one market in this design is a two-node cluster plus small scheduled jobs.	Hardware, licences and people are fixed costs whether the pipeline is busy or idle.
What comes next	Habeeb's point from the session: within one to two years most AI use cases will need all of this data. The AI brief, product suggestions and natural-language questions in the demo run on the same tables with no new platform.	Another migration later, when the AI use cases arrive.
Already proven at SCB	CIB and Financial Markets already move data into Databricks after governance clearance, in the same isolated Azure workspace. The pattern, the network and the controls exist.	A real-time ODS integrating directly from source would be a new pattern either way (Dhas).

What stays on-premise, deliberately

Clear customer data
Names, IDs and account numbers are tokenized before anything leaves the bank. The cloud only ever holds tokens.

Solace and the source systems
No code changes for source teams. The bridge reads what they already publish.

The RM-facing API
SpringWatch keeps serving RMs. Databricks pushes signals to it; it joins tokens back to real customers.

GDP / EDMP
Untouched. The ODS sits beside it and can feed it, not the other way round.

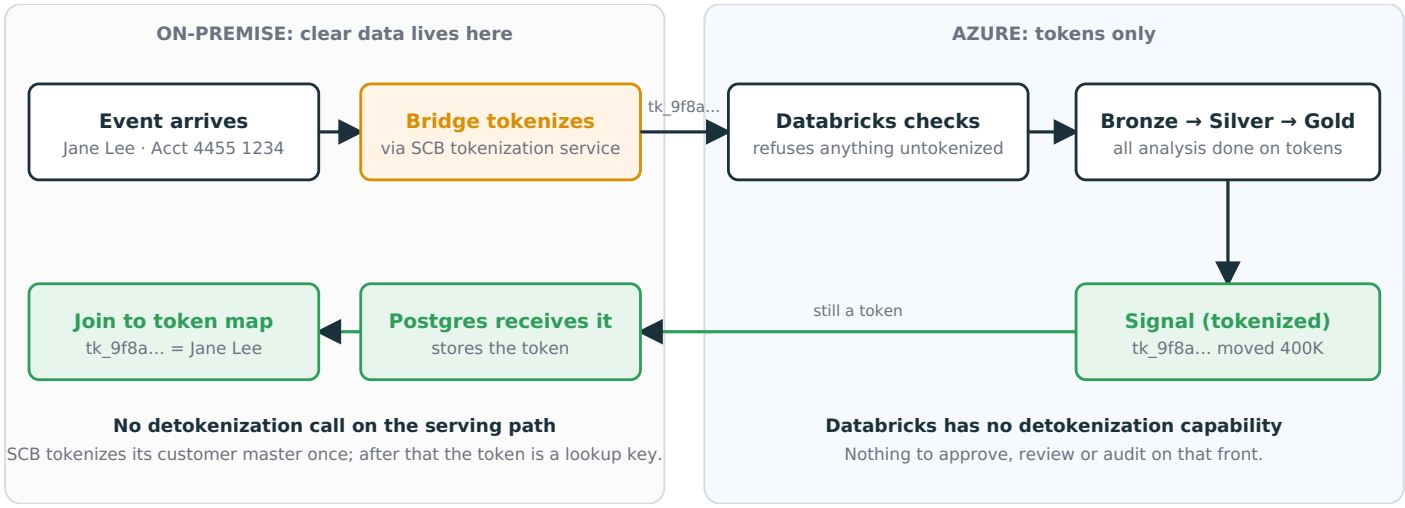
Honest trade-offs

- **Explore the network path** from Databricks to the on-prem Postgres and APIs. If a direct path is not available, the Kafka signal topic already carries everything and becomes the only transport (page 5).
- **Portability is a requirement, not a hope.** For markets where cloud is not permitted (China), the pipeline code is written in open Spark so it runs on SCB's own engine. The plan includes proving that, not assuming it.

KEEPING CUSTOMER DATA SAFE

Tokens leave the bank. Names do not.

The most important design decision, and the one the security review will ask about first. Because it is the same token every time for the same customer, Databricks can still do all the analysis (baselines, counting contacts, joining accounts) without ever knowing who the customer is.



The controls, in one list

CONTROL	WHAT IT DOES	WHERE IT LIVES
Tokenize before leaving on-prem	The bridge replaces every configured customer field with a token from SCB's own tokenization service. If a required field is missing, the record is rejected, not passed through.	Bridge (TokenizePII)
Only tokenized topics are mirrored	The replication allow-list matches only `*.tokenized` topic names. The bridge's error queue is named `*.CLEAR` and is explicitly excluded, so clear text can never be mirrored by mistake.	MirrorMaker 2 config
Fail-closed guard on landing	Databricks checks a "tokenized" header and that every customer field looks like a token. Anything else is quarantined as a reference and a hash, never the content.	Bronze job
Rules and tests keep configs in step	An automated test fails if the bridge's field list and the Databricks guard's field list ever differ.	CI (42 tests)
Least privilege on-prem	The Databricks database login can write only to a staging table, the signals mirror and a watermark. It cannot read the token map or the customer master.	Postgres roles
Residency by market	Each market has its own schemas and storage, so per-country rules (and China's no-cloud rule) are enforced at the storage boundary.	Unity Catalog

One trade-off to state plainly: because Bronze is already tokenized, the raw clear-text record cannot be replayed from Databricks. The on-prem Kafka topic becomes the record of truth for replay, and its retention should be sized accordingly (we suggest at least 7 days).

WHAT IS IN THE BOX

Three deployable parts, one repository

Everything described in this plan is already written and tested. The repository `scb-wrb-ods-ingestion` has three folders, one per team. Page 9 lists every file and when to use it.

`bridge/`

Kafka / Integration team

- **Three connector configs** (core banking, customer profile, RM outcomes): Solace queue → Kafka, with the tokenization step built in.
- **TokenizePII**: a small Java plug-in for Kafka Connect. Calls the bank's tokenization service in batches, caches results, refuses records it cannot tokenize.
- **MirrorMaker 2 config**: copies tokenized topics to Azure; copies one signal-events topic back.

`databricks/`

Databricks team

- **Asset bundle**: dev / UAT / prod targets. One `deploy` command creates everything below.
- **Setup SQL**: catalog, Bronze / Silver / Gold tables, rules seed, views for Genie and the console.
- **Six jobs**: Bronze ingest, Silver + Gold streams, baseline refresh (15 min), housekeeping (5 min), serving streams, nightly reconciliation.
- **Nine alerts** on latency, quarantine, stalls and drift.

`onprem-postgres/`

Database team

- **001_schema.sql**: the signals mirror table, a staging table, a sync watermark, an outcomes outbox, and two roles (one for Databricks, one for the API).
- **002_token_resolution.sql**: the token-to-customer map and the view the RM API reads, with clear names joined in on-prem.
- A view that flags tokens that could not be resolved, so the map can be kept current.

How a signal is decided (the rules WRB owns)

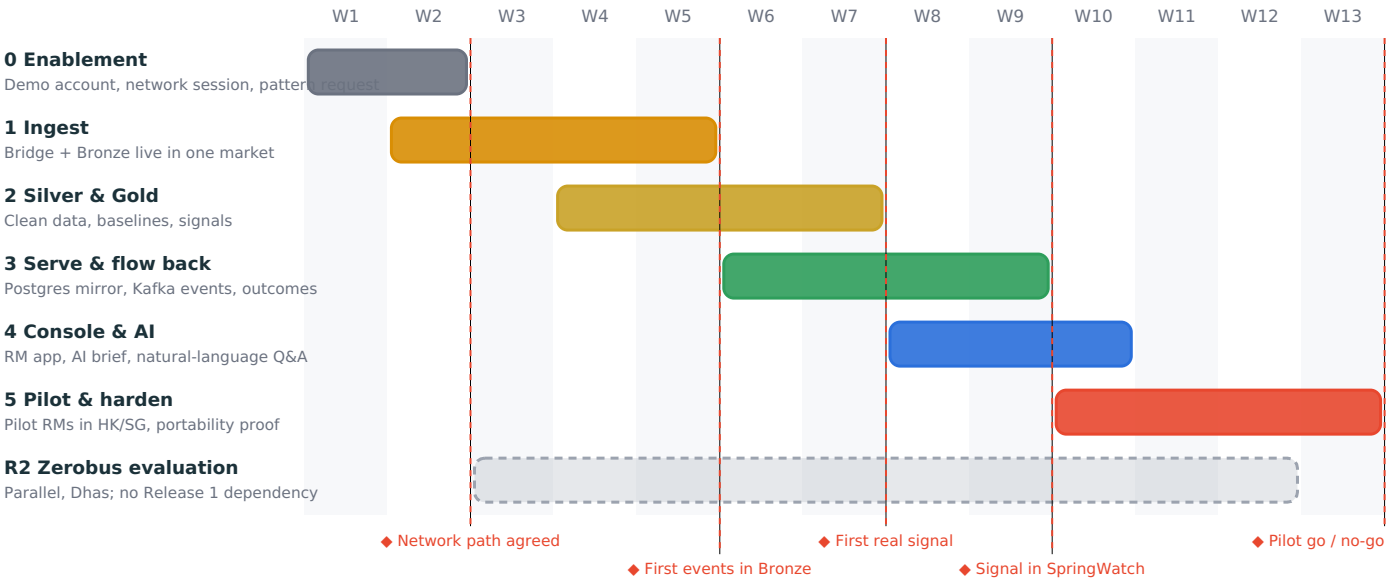
STEP	PLAIN-ENGLISH RULE	WHERE TO CHANGE IT
Is it significant?	Ignore anything under 50K USD. Always flag 250K USD and above. In between, flag if it is more than 30× the client's average transaction over the last 30 days (needs at least 5 past transactions). Separately, flag any client whose outflows in the past hour exceed 10M USD.	`gold.signal_rules` table, no code change
What kind of signal?	Money in → an opportunity to pitch a product. Money out → verify intent; flagged as possible fraud if the counterparty is new, the amount is 100× normal, or it is an hourly burst.	`rules.py` (tested)
Should the RM be told?	Not if the client is do-not-contact, vulnerable, has an open complaint, has already been contacted 3 times in 48 hours, got the same signal in the last hour, or has no RM. Possible-fraud signals skip the first three (protecting the client comes first). Suppressed signals are still recorded so team leads can tune the rules.	`gold.signal_rules` + customer flags
When does it expire?	4 hours after creation unless an RM acts on it.	`signal_ttl_hours`

All thresholds are in USD so one rules table serves every market. The values above are the demo defaults; WRB rule owners set the real ones.

THE PLAN

Thirteen weeks to a live pilot

Indicative. Phases overlap because different teams own them. The red diamonds are the moments where we know we are on track.



PHASE	WHAT GETS DONE	DONE WHEN	OWNER
0 Enablement W1–2	Demo account and tutorials for WRB engineers; joint network session; pattern-approval request raised; tokenization scheme and token format agreed.	Network contact named; the three connectivity paths mapped; token regex confirmed.	Sid · Dhas/Habeeb · tokenization team
1 Ingest W2–5	Bridge deployed for the core-banking queue in one market; topics mirrored; Bronze job running with the tokenization guard.	Events land in Bronze within seconds; security team confirms tokenization; quarantine stays empty.	Kafka team · Databricks
2 Silver & Gold W4–7	Customer-profile bridge; Silver clean-up; baselines; rules table seeded; first signals raised on real (tokenized) data.	Rules reproduce the demo; P95 time from movement to signal under 5 minutes.	Databricks · WRB rule owners
3 Serve & flow back W6–9	Postgres schema and token map; signals mirrored on-prem; events on the Kafka topic; RM outcomes flowing back.	A signal appears in SpringWatch with the client's real name within seconds; outcome closes the loop.	DBA · Databricks · SpringWatch team
4 Console & AI W8–10	Databricks App with RM, team-lead, fraud and architect views; AI brief grounded in the data; natural-language questions via Genie.	RM user-acceptance test passed in the pilot market.	Databricks · WRB product
5 Pilot & harden W10–13	Pilot RMs in HK/SG; runbooks; alerts tuned; generated pipeline code run on SCB's own Spark to prove portability.	Pilot response rate and latency reported; portability run passes; go/no-go for second market.	WRB · Databricks

DEPLOYMENT GUIDE · 1 OF 2

Before you start, and the on-prem bridge

Follow the steps in order. Each step says who does it. Commands are copied from the repository README; nothing here needs to be typed from memory.

A Prerequisites (everyone, week 1)

☐	**Tokenization team**	Confirm the batch endpoint, a **deterministic** scheme name (same input → same token, always) and the token format. Edit: `bridge/solace-source-connector*.json` (service.url, service.scheme) · `databricks/src/config/topics.yml` (token_pattern)
☐	**Network (SCB cloud platform)**	Three paths: on-prem Kafka → Azure (exists for mirroring); Databricks → on-prem Postgres port 5432 (new, needs allow-list); Databricks Apps → on-prem APIs (separate thread).
☐	**Architecture (Habeeb)**	Raise pattern approval for "Databricks sources from / writes to a business-application database".
☐	**Databricks admin**	Secret scope `wrb-ods` with: `kafka_bootstrap`, `kafka_oauth_client_id`, `kafka_oauth_client_secret`, `kafka_oauth_token_endpoint`, `kafka_oauth_scope`, `pg_host`, `pg_port`, `pg_database`, `pg_user`, `pg_password`. Service principal `wrb-ods-ingest-sp`. SCB root CA uploaded to the certs volume.
☐	**Kafka team**	Decide: SCB-run Kafka on Azure behind Private Link (default in configs) or Azure Event Hubs' Kafka endpoint. Edit: `bridge/mirror-maker2.properties` (azure.bootstrap.servers, OAuth scope)

B On-prem bridge (Kafka / Integration team, weeks 2–3)

1 Build the tokenization plug-in and install it on the Kafka Connect workers.
Files: `bridge/tokenize-smt/pom.xml`, `bridge/tokenize-smt/src/.../TokenizePII.java`

```
cd bridge/tokenize-smt && mvn package
cp target/tokenize-smt-0.1.0.jar /opt/kafka/connect-plugins/ && systemctl restart kafka-connect
```

2 Create the topics. The error-queue topic must end in `.CLEAR` and have restricted access; it is the only place clear text can appear.
Topic names are in: `bridge/solace-source-connector.json` (kafka.topic, errors.deadletterqueue.topic.name)

```
kafka-topics --create --topic wrb.ods.hk.core_banking.tokenized --partitions 12 --replication-factor 3
kafka-topics --create --topic wrb.ods.hk.core_banking.bridge_dlq.CLEAR --partitions 3 --replication-factor 3
```

3 Deploy the connector and check one record: customer fields are tokens, `_tokenization_scheme` is stamped, header `x-pii-tokenized=true` is present.
File: `bridge/solace-source-connector.json`

```
curl -X POST -H "Content-Type: application/json" --data @solace-source-connector.json http://connect:8083/connectors
kafka-console-consumer --topic wrb.ods.hk.core_banking.tokenized --from-beginning --max-messages 1 --property print.headers=true
```

4 Start MirrorMaker 2 and confirm the tokenized topic appears on the Azure cluster **and the `.CLEAR` topic does not**.
File: `bridge/mirror-maker2.properties`

```
connect-mirror-maker.sh mirror-maker2.properties
```

5 Repeat step 3 for the other two queues.
Files: `bridge/solace-source-connector-customer-profile.json` (week 4) · `bridge/solace-source-connector-signal-outcomes.json` (week 7)

If something goes wrong: records the bridge cannot tokenize go to the `.CLEAR` queue and are logged (without the content). Fix the cause, then replay from that queue. Nothing reaches Azure until it is tokenized.

DEPLOYMENT GUIDE · 2 OF 2

Databricks and the on-prem database

C Databricks (Databricks team + WRB engineers, from week 3)

1 Validate and deploy the bundle to dev. This creates all jobs, the SQL warehouse and the alerts; nothing starts running yet.
Files: `databricks/databricks.yml` (targets, hosts) · `databricks/resources/*.yml` (jobs, alerts)

```
cd databricks
databricks bundle validate -t dev
databricks bundle deploy -t dev --var market=hk --var storage_root=abfss://wrbods@scbadls.dfs.core.windows.net --var ingest_sp=<app-id>
```

2 Create the catalog, schemas and tables (Bronze, Silver, Gold, serving ops). Safe to re-run.
Runs: `src/setup/00_create_catalog.sql` → `01_create_silver_gold.sql` → `02_create_serving_ops.sql`

```
databricks bundle run wrb_ods_bronze_setup -t dev
```

3 Load a first FX reference (a single `USD, 1.0` row is enough to test) into `wrb_ods.hk_silver.ref_fx_rates`, then schedule the bank's daily FX feed into it.

4 Start Bronze ingestion. Watch `wrb_ods.hk_ops.bronze_ingest_audit`: `max_latency_ms` should be well under 60 000 and `rows_quarantined` should be 0.
Runs: `src/ingest/bronze_ingest.py` + `guard.py` with `src/config/topics.yml` (job: `resources/bronze_ingest_job.yml`)

```
databricks bundle run wrb_ods_bronze_ingest -t dev
```

5 Build the first baseline, then start the Silver and Gold streams. The baseline job then runs itself every 15 minutes; housekeeping every 5.
Runs: `src/gold/baseline_refresh.py`, `src/silver/silver_transactions.py`, `silver_dimensions.py`, `src/gold/signal_detection.py`, `signal_housekeeping.py` (job: `resources/silver_gold_jobs.yml`)

```
databricks bundle run wrb_ods_gold_baseline_refresh -t dev
databricks bundle run wrb_ods_silver_gold_streams -t dev
```

6 Once the Postgres schema exists (D below), start the serving streams.
Runs: `src/serving/postgres_sink.py`, `kafka_emit.py`, `outcomes_ingest.py`; nightly `reconcile.py` (job: `resources/serving_jobs.yml`)

```
databricks bundle run wrb_ods_serving_streams -t dev
```

7 Promote: the same commands with `-t uat` and `-t prod`. The bundle runs as the service principal in those targets.

D On-prem Postgres (SCB DBA, week 6)

1 Create the serving schema, roles and tables as the ODS owner.
Files: `onprem-postgres/001_schema.sql`, `002_token_resolution.sql`

```
psql -d ods -f onprem-postgres/001_schema.sql
psql -d ods -f onprem-postgres/002_token_resolution.sql
```

2 Populate `wrb_ods.customer_token_map` and `account_token_map` by tokenizing the customer master once with the same scheme the bridge uses. Schedule it to re-run for changed customers.

3 Point `v_rm_signals_active_resolved` at the real customer-master table name (the script assumes `ods.customer_master`). Grant the SpringWatch API read on that view only.

4 Have the SpringWatch team write RM outcomes to `wrb_ods.signal_outcomes_outbox`, and schedule the outbox publisher to send unpublished rows to Solace.

E Smoke test (everyone, 20 minutes)

1 Publish the demo 400K HKD transfer to the Solace queue. Expect: one row in Bronze, one in Silver, one in `gold.significant_balance_events`, one in `gold.rm_signals` with `status = 'CREATED'`.

2 Within seconds: the same signal in `wrb_ods.rm_signals` on-prem, showing the client's real name through `v_rm_signals_active_resolved`, and a `signal.created` event on the on-prem Kafka topic.

3 Insert an outcome in the outbox. Expect: a row in `gold.signal_outcomes`, then `status = 'ACTIONED'` in both Gold and Postgres and a `signal.actioned` event.

4 Publish a record directly to the Azure topic **without** the tokenized header. Expect it in `bronze_quarantine` (as a hash) and the quarantine alert to fire. This proves the guard.

DEPLOYMENT PACKAGE

Every file, who uses it, and when

The package `scb-wrb-ods-deployment.zip` contains this guide, the architecture overview (`.md`) and the `scb-wrb-ods-ingestion` repository (with its technical `README.md`). Paths below are inside the repository. Edit only the files marked "edit"; the rest deploy as-is. Step letters refer to pages 7–8.

FILE	OWNER	WHEN	WHAT IT IS FOR
bridge/ — on-prem Kafka Connect and MirrorMaker			
`tokenize-smt/pom.xml` `tokenize-smt/src/main/java/.../TokenizePII.java`	Kafka team	B1 · week 2	Build the tokenization plug-in jar. Deploy as-is.
`solace-source-connector.json` **edit**	Kafka team	A, B2, B3 · week 2–3	Core-banking queue → Kafka with tokenization. Set Solace host/VPN/queue, tokenization URL and scheme.
`solace-source-connector-customer-profile.json` **edit**	Kafka team	B5 · week 4	Customer profile / CRM preference queue (RM assignment, do-not-contact, complaints).
`solace-source-connector-signal-outcomes.json` **edit**	Kafka team	B5 · week 7	RM outcome events from the on-prem outbox back to Databricks.
`mirror-maker2.properties` **edit**	Kafka team	A, B4 · week 3	Mirror tokenized topics to Azure; one signal-events topic back. Set both clusters' addresses and credentials.
databricks/ — asset bundle (deploy with one command)			
`databricks.yml` **edit**	Databricks	C1 · week 3	Workspace hosts for dev/uat/prod, cluster size, alert e-mail, service principal.

`resources/bronze_ingest_job.yml`
`resources/silver_gold_jobs.yml`
`resources/serving_jobs.yml`
	Databricks	C1	Job, warehouse and alert definitions. Deploy as-is; add a task per new topic.

`src/setup/00_create_catalog.sql`
`src/setup/01_create_silver_gold.sql`
`src/setup/02_create_serving_ops.sql`
	Databricks	C2	Catalog, schemas, all tables, rules seed, views, grants. Idempotent.

`src/config/topics.yml` **edit**	Databricks	A, C4	Topic → table map and the tokenization guard (token regex, PII fields). Must match the connector configs; a test enforces it.
`src/ingest/bronze_ingest.py`, `guard.py`	Databricks	C4 · week 3–5	Bronze streaming job with the fail-closed tokenization guard.
`src/silver/silver_transactions.py`, `silver_dimensions.py`, `dq.py`	Databricks	C5 · week 4–7	Clean-up, dedupe, FX to USD, customer/account dimensions, data-quality rules.
`src/gold/rules.py`, `baseline_refresh.py`, `signal_detection.py`, `signal_housekeeping.py`	Databricks · WRB rule owners	C5 · week 5–7	Baselines, significance, suppression, signals, expiry. Threshold values live in the `signal_rules` table, not in code.
`src/serving/postgres_sink.py`, `kafka_emit.py`, `outcomes_ingest.py`, `reconcile.py`, `sink_utils.py`	Databricks	C6 · week 6–9	Push signals to on-prem Postgres and Kafka; pull outcomes back; nightly reconciliation.
`src/tests/` (42 tests)	Databricks CI	every change	Run with `python -m unittest` before any deploy. Catches config drift between bridge, guard, rules and DDL.
onprem-postgres/ — SpringWatch ODS			
`001_schema.sql`	DBA	D1 · week 6	Signals mirror, staging, watermark, outcomes outbox, two roles. Deploy as-is.
`002_token_resolution.sql` **edit**	DBA	D1, D3 · week 6	Token maps and the resolved view for the RM API. Point the view at the real customer-master table.

RUNNING IT

How we know it is working, and what still needs deciding

Dashboards and alerts (all created by the bundle)

QUESTION	WHERE TO LOOK	ALERT FIRES WHEN
How fast is money movement → signal?	`signal_latency_ms` on every signal; Architect view; `signal_funnel_daily` P50/P95	P95 over 5 minutes in the last 15 min
Is anything untokenized getting through?	`bronze_quarantine` (references and hashes only)	Any row in the last 15 min
Is Bronze keeping up with Kafka?	`bronze_ingest_audit.max_latency_ms`	Over 60 s
Is the baseline fresh?	`client_30d_baseline.as_of_ts`	Older than 45 min
Are signals reaching Postgres?	`postgres_sink_audit.sink_lag_ms`	Lag over 60 s, or no batch for 30 min
Do Gold and Postgres agree?	`postgres_sink_reconciliation` (nightly counts and checksums)	Any mismatch
Are RMs acting on signals? Are rules too noisy?	`signal_funnel_daily`: created vs suppressed vs actioned, by team	Team-lead review, weekly
Are there customers we cannot name on-prem?	`v_unresolved_signal_tokens` (Postgres)	SCB-side alert

Decisions for Habeeb, Dhas and Mario

DECISION	OPTIONS AND OUR RECOMMENDATION
1. Where is the ODS "truth"?	(a) Databricks processes, Gold is mirrored to on-prem Postgres (Dhas's current lean; what this plan builds). (b) On-prem first, Databricks copies from it. The build supports both; (a) is faster for the latency-sensitive path and keeps one reconciliation. **Recommend (a).**
2. Kafka on Azure	SCB-run cluster behind Private Link, or Event Hubs' Kafka endpoint. Same Databricks code either way. **Recommend whichever the Kafka programme can support by week 3.**
3. Pattern approval	Databricks writing to a business-application database. Scope is narrow: one login, three tables, no access to customer data. **Needed by week 6.**
4. Zerobus	Evaluate in parallel against the Solace client model and source-team appetite. **No Release 1 dependency.**

Risks and what we do about them

RISK	MITIGATION ALREADY IN THE DESIGN
Kafka on Azure is not ready in time	Event Hubs as interim; identical pipeline code.
Tokenization service is slow	Batch calls and caching in the bridge; if P99 is above ~50 ms, move tokenization in front of the connector.
JDBC to on-prem Postgres is blocked	The signal-events topic already carries the full signal; an on-prem consumer can load Postgres from it with the same upsert statement.
Tool-calling from Databricks to on-prem APIs stays blocked	AI brief grounded only in Databricks data for Release 1; API calls in a later release.
Code drifts away from open Spark	Portable core; a CI job runs it on open-source Spark; portability proof in week 11.
Rules too noisy, RMs ignore signals	Rules are data WRB can change without a release; suppressed signals are recorded for tuning.

REFERENCE

Words used in this plan

TERM	PLAIN MEANING
ODS	Operational Data Store. A place where current data from several systems is kept together so questions like "how much left this account in the last hour?" can be answered without asking the core banking system.
Solace / Kafka	Message buses. Systems publish events (a payment happened) and others subscribe. SCB uses Solace today; Kafka is being rolled out. The bridge connects the two.
Bridge / Kafka Connect	A small service that reads from Solace, tokenizes customer data, and writes to Kafka. No changes to the systems that publish the events.
Tokenization	Replacing a sensitive value (a name, an account number) with a meaningless stand-in. Deterministic tokenization always gives the same stand-in for the same value, so data can still be joined and counted.
MirrorMaker 2	Copies Kafka topics from one place to another; here, from on-prem to Azure (tokenized only) and one topic back.
Unity Catalog	Databricks' catalogue of all tables, with who may see what and where each piece of data came from (lineage).
Delta table	The table format Databricks uses. Supports exact replay, time travel and a change feed (a list of every insert and update).
Bronze / Silver / Gold	Raw copy / cleaned data / business answers. A convention for layering data so each step is simple and auditable.
Structured Streaming	The part of Spark that processes events as they arrive, in small batches every couple of seconds, with checkpoints so nothing is lost or counted twice.
Baseline	Each client's own normal: average and typical transaction size over the last 30 days, per direction (in / out).
Suppression	Deciding not to tell the RM even though the movement was unusual: do-not-contact, vulnerable client, open complaint, already contacted three times, duplicate.
Lakebase	Databricks' fast transactional database, used for the RM console and as memory for the AI brief. A cloud-side parallel to the on-prem Postgres, fed from the same Gold tables.
Change Data Feed (CDF)	The list of every row inserted or updated in a Delta table. The serving layer reads it to push changes to Postgres and Kafka.
Asset bundle	A folder describing every job, table and alert, so the whole platform deploys with one command and lives in Git.
Genie	Databricks' natural-language query tool. "Which clients moved more than twice their normal this week?" becomes SQL.
Zerobus	A newer Databricks option where a source system writes straight into Delta without a Kafka hop. Release 2 evaluation only.
SpringWatch	WRB's on-prem ODS layer on Postgres and the API that RM applications read.

Prepared by Siddharth Pandey, Lead Solutions Architect, Databricks, following the 21 Oct 2026 WRB Architecture session with Habeeb Rahman Yousuff, Mario Wiradinata, Dhas Durai and Shashikiran Rao. Code and configuration referenced here are in the `scb-wrb-ods-ingestion` repository (34 files, 42 automated tests).